Experiment 2 Github commands:-

Theory

Git is a distributed version control system used to track changes in source code and maintain different versions of a project. It allows developers to manage code, revert previous changes, and work collaboratively.

GitHub is a cloud-based platform used to host Git repositories and facilitate collaboration among developers. It provides features such as code sharing, branching, commits, and project management.

Procedure

1. Install Git and verify its installation using git --version.
2. Configure the Git username using git config --global user.name "Your Name".
3. Configure the email using git config --global user.email "your@email.com".
4. Check the configuration using git config --list.
5. Create a project folder using mkdir GitProject.
6. Navigate to the folder using cd GitProject.
7. Initialize the repository using git init.
8. Check the repository status using git status.
9. Create a README file using echo "# My First Git Project" > README.md.
10. Add files to the staging area using git add ..
11. Commit the changes using git commit -m "Initial commit".
12. Rename the branch to main using git branch -M main.
13. Create a new repository on GitHub.
14. Connect the local repository to GitHub using git remote add origin <repository-URL>.
15. Upload the project using git push -u origin main.
16. Verify that the project files are successfully uploaded to GitHub.

Experiment 3 Jenkins:-

Theory

Jenkins is an open-source automation server used for Continuous Integration (CI) and Continuous Delivery (CD). It helps automate software development tasks such as building, testing, and deploying applications.

Jenkins integrates with tools like Git and Maven to automate the software development process. It uses plugins to extend its functionality and supports automated builds whenever code changes are made.

Procedure

1. Install the prerequisite Java JDK on the system.
2. Download the Jenkins LTS Windows installer from the official Jenkins website.
3. Run the installer and complete the installation.
4. Configure Jenkins to use an available port (default port: 8080).
5. Start the Jenkins service.
6. Open a web browser and navigate to http://localhost:8080.
7. Unlock Jenkins using the initial administrator password.
8. Install the suggested plugins and create an administrator account.
9. Open the Jenkins dashboard and click on New Item.
10. Enter a project name and select Freestyle Project.
11. Configure the build step to execute a command that prints Hello World.
12. Save the project and click Build Now.
13. Open Console Output to verify the build execution.

Result

Jenkins was successfully installed and configured, and a sample Hello World build was executed successfully.

Experiment 4 Integration of Jenkins and github:-

1. Theory

Jenkins is an open-source automation server used for Continuous Integration (CI) and Continuous Delivery/Deployment (CD). It automates software development tasks such as building, testing, and deploying applications.

GitHub is a cloud-based platform used to store and manage source code using Git version control.

By integrating Jenkins with GitHub, developers can automate the process of retrieving source code, building applications, and running automated tests. This reduces manual effort, improves software quality, and helps identify errors early in development.

CI/CD Pipeline Stages:

1. Checkout: Retrieves source code from the GitHub repository.
2. Build: Executes the application and prepares it for testing.
3. Test: Runs automated test cases to identify errors.
4. Result: Displays whether the pipeline completed successfully or failed.

2. Procedure

1. Install Java JDK, Git, and Jenkins on the system.
2. Open Jenkins using http://localhost:8080.
3. Unlock Jenkins using the initial administrator password.
4. Install the suggested plugins and create an administrator account.
5. Create a new public repository on GitHub.
6. Create a sample application and automated test files.
7. Push the project files to GitHub using Git commands.
8. Create a Jenkinsfile containing the pipeline stages: Checkout, Build, and Test.
9. Push the Jenkinsfile to the GitHub repository.
10. Open Jenkins and select New Item.
11. Enter a project name and select Pipeline.
12. Under Pipeline configuration, select Pipeline script from SCM.
13. Select Git as the SCM and enter the GitHub repository URL.
14. Specify the branch as */main and script path as Jenkinsfile.
15. Save the configuration and click Build Now.
16. Open Console Output to monitor the pipeline execution.
17. Verify the build and test results.
18. If successful, Jenkins displays Finished: SUCCESS.

Experiment 5 Kubernetes:-

Theory

Kubernetes is an open-source container orchestration platform used to automate the deployment, management, and scaling of containerized applications.

Minikube is a tool that runs a local, single-node Kubernetes cluster on a computer. It is mainly used for learning, testing, and experimenting with Kubernetes.

Kubernetes uses different components such as Pods, Deployments, and Services to manage applications. Pods run containers, Deployments manage Pods, and Services provide network access to applications.

The kubectl command-line tool is used to interact with and manage the Kubernetes cluster.

Procedure

1. Install Docker Desktop, Minikube, and kubectl on the Windows system.

2. Start Docker Desktop and ensure that the Docker engine is running properly.

3. Open PowerShell and start the Minikube cluster using Docker as the driver.

4. Check the status of the Minikube cluster and verify that the Kubernetes node is in the Ready state.

5. Create a Kubernetes Deployment named hello-node using a sample container image. The Deployment creates a Pod and runs the application inside it.

6. Verify that the Deployment has been created successfully and check the status of the Pod. Ensure that the Pod is in the Running state.

7. Expose the Deployment using a Kubernetes Service of type LoadBalancer so that the application can be accessed through a network connection.

8. Check the Service details to verify that the application has been exposed successfully.

9. Access the deployed application using the Minikube service option. The application opens in a web browser and displays its response.

10. Open the Kubernetes Dashboard using Minikube to view and monitor cluster resources such as Pods, Deployments, and Services.

11. After completing the experiment, delete the created Service and Deployment to clean up the resources.

12. Stop the Minikube cluster. If required, delete the local cluster completely.